

Reviewer Pack — Minimal Alexander-Defect Invariant and Communication Complex...

Entry 7045cac6460e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 06:58:25 UTC

Minimal Alexander-Defect Invariant and Communication Complexity Rank

Entry ID: 7045cac6460e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 06:58:25 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Alexander Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every boolean function f with n variables, the Alexander-defect invariant of its associated matroid M_f is linearly correlated with its communication complexity rank $r(f)$, such that $A(M_f) = \Theta(r(f))$.

Rationale (proposer's reasoning):

The Alexander-defect invariant provides a topological measure for the complexity of embedding a graph into a surface. This conjecture suggests that this topological property could be related to the communication complexity, potentially revealing new insights into both fields.

Taxonomy category: Alexander_defect_invariant (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 570ebfdc79d8c4e2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The Alexander-defect invariant $A(M_f)$ of matroid M_f associated with boolean function f will be considered supported if Pearson correlation coefficient ≥ 0.7 AND p-value ≤ 0.05 for a sample size of $n=30$ and $n \leq 40$ instances.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Alexander-invariant AND matroid AND communication complexity rank - communication complexity rank AND Alexander Theory AND boolean function - boolean function AND Alexander-defect invariant AND matrix rank

Top relevant hits considered: - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/0903.3848v1] Join-irreducible Boolean functions - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.5s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_matroid(M):
        rank = len(set(tuple(sorted(subset)) for subset in M if all(M[i] == 1 for i in subset)))
        return rank

    def compute_alexander_defect(M):
        n = len(M)
        matroid_matrix = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i+1, n):
                if all(M[i] == M[j] for k in range(2**n)):
                    matroid_matrix[i][j] = 1
        rank = compute_matroid(matroid_matrix)
        return rank

    def communication_complexity(f):
        # Simplified version of communication complexity calculation
        n = len(f)
        return n * (n - 1) // 2
```

```

metric_name = "alexander_defect_vs_communication_complexity"
instances_tested = 0
total_alexander_defect = 0
total_communication_complexity = 0
n_max = 5

for n in [5, 10, 15, 20, 30, 40]:
    if n > n_max:
        n_max = n

    for _ in range(5):
        f = generate_boolean_function(n)
        M = [[i] for i in range(n)]
        alexander_defect = compute_alexander_defect(M)
        communication_complexity_val = communication_complexity(f)

        total_alexander_defect += alexander_defect
        total_communication_complexity += communication_complexity_val
        instances_tested += 1

mean_alexander_defect = Fraction(total_alexander_defect, instances_tested)
mean_communication_complexity = Fraction(total_communication_complexity, instances_tested)

correlation_coefficient = (instances_tested * mean_alexander_defect * mean_communication_complexity -
                           total_alexander_defect * total_communication_complexity) / \
    math.sqrt((instances_tested * mean_alexander_defect**2 - total_alexander_defect**2) *
              (instances_tested * mean_communication_complexity**2 - total_communication_complexity**2))

p_value = 1.0 # Placeholder for actual p-value calculation

conjecture_holds = correlation_coefficient >= Fraction(7, 10) and p_value <= Fraction(5, 100)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": metric_name,
    "metric_value": float(correlation_coefficient),
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif sum(1 for result in results if not result["conjecture_holds"]) / len(results) >= 0.8:
    first_failing_seed = next(result["seed"] for result in results if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support conditions could not be met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12231
2	preregistration	ollama_remote	glm4:latest	0	0	9618
3	novelty	ollama_remote	glm4:latest	0	0	8244
4	novelty	ollama_remote	glm4:latest	0	0	9293
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23440
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15609
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12289
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	40234
9	judge	ollama_remote	glm4:latest	0	0	52189

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 183147 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7045cac6460e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7045cac6460e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/7045cac6460e.tar.gz` (if generated)